高级操作系统

名词解释

1. **进程 p508**
   - 一个执行中程序的实例（process is a running program object)
2. **虚拟内存 p559**
   - virtual memory，虚拟内存是硬件异常、硬件地址翻译、主存、磁盘文件和内核软件的完美交互，它为每个进程提供了一个大的、一致的和私有的地址空间。
3. **I/O重定向 p637**
   - 允许用户将磁盘文件和标准输入输出联系起来
4. **碎片 p591**
   1. 堆里的虽然有未使用的内存但不能用来满足分配请求，这些不能分配的块叫做碎片
5. **内部碎片 p591**
   - 内部碎片是处于区域内部或页面内部的存储块。占有这些区域或页面的进程并不能使用这个存储块。而在进程占有这块存储块时，系统无法利用它。直到进程释放它，或进程结束时，系统才有可能利用这个存储块。
6. **外部碎片 p591**
   - 外部碎片是出于任何已分配区域或页面外部的空闲存储块。这些存储块的总和可以满足当前申请的长度要求，但是由于它们的地址不连续或其他原因，使得系统无法满足当前申请。
7. **inode**
   - 用来记录文件系统对象的一些元信息数据，简称“索引节点”。本质上是一个结构体。（ext4为256B）
8. **线程 p691**
   - thread 线程就是运行在进程上下文中的逻辑流。
9. **互斥 p701**
   - 想要确保每个线程在执行它的临界区中的指令时，拥有对共享变量的互斥的访问。
10. **死锁 p721**
    - 指的是一组线程被阻塞了，等一个永远也不会为真的条件。
11. **竞争 p719**
    - 竞争是当一个程序的正确性依赖于一个线程要在另一个线程达到y点之前到达它的控制流中的x点时发生的。通常发生竞争是因为程序员假定线程将按照某种特殊的轨迹线穿过执行状态空间，而忘记了另一种准则规定：多线程的程序必须对任何可行的轨迹线都正确工作。
12. **文件系统**
    - 文件系统是用于明确存储设备或分区上的文件的方法和数据结构；即在存储设备上组织文件的方法。操作系统中负责管理和存储文件信息的软件机构称为文件管理系统，简称文件系统。由三部分组成：文件系统的接口，对对象操纵和管理的软件集合，对象及属性。
13. **无锁化**
    - 无锁化编程特指在多线程编程的时候，对线程间共享数据的并发修改不使用锁， 而采用基于硬件提供的原子操作能力来修改共享数据，进而提升性能，减少使用锁带来的互斥开销。
14. **并发 p509**
    - 一个逻辑控制流在时间上与另一个逻辑控制流重叠称为并发 concurency
15. **并行 p509**
    - 两个逻辑控制流并发地运行在不同的处理器或者计算机上称这两个流为并发
16. **异常 p502**

- exception，异常就是控制流中的突变，用来响应处理器状态中的某些变化。

17. **信号量 p702**
    - 信号量s是具有非负整数值的全局变量，只能由两种特殊的操作来处理，这两种操作称为P和V。是解决同步不同执行线程问题的方法。

问答题

1. **操作系统的虚拟内存机制中，软硬件是如何配合工作的？请简要陈述。**
    1. CPU生成一个虚拟地址（VA），并把它传送给芯片上的内存存储单元（MMU）
    2. MMU生成PTE地址（PTEA），并从缓存/主存中请求这个PTE
    3. 高速缓存/主存向MMU返回PTE
    4. MMU构造物理地址，并把它传送给缓存/主存
    5. 缓存/主存返回所请求的数据字给CPU
    6. 参考链接
2. **操作系统一般可以分为哪三个主要模块？（Three Easy Pieces）** reference
    1. 虚拟化
        - 进程
        - 虚拟内存
    2. 并行性
    3. 持久性
3. **操作系统对上提供的抽象（Abstract）有什么优点？请给出 3 个或更多经典抽象的例子。**
    1. 操作系统

       - 应用程序和硬件之间的一层软件，防止硬件被失控的程序滥用
       - 向应用程序提供简单一致的机制来控制复杂而大相径庭的低级硬件设备

    2. 进程 p508
        - 一个独立的逻辑控制流，它提供一个假象，好像我们独占地使用处理器
        - 一个私有的地址空间，它提供一个假象，好像我们的程序独占地使用内存系统
    3. 虚拟内存 p559
        - 它将主存看成是一个存储在磁盘上的地址空间的高速缓存，在主存中只保存活动区域，并根据需要在磁盘和主存之间来回传送数据，高效地使用主存
        - 为每个进程提供了一致的地址空间，从而简化了内存管理
        - 保护了每个进程的地址空间不被其他进程破坏
    4. 文件
        - 为应用程序提供了一个统一的视觉来看待系统中的I/O设备。同一个程序可以使用不同的磁盘技术的不同系统运行
4. **malloc 内存分配需要考虑哪两个指标？他们之间有什么关系？p590~p591**
    1. 最大吞吐率：
        - 吞吐率定义为每个单位时间里完成的请求数。如一个分配器1秒内完成500个分配请求和500个释放请求，那么它的吞吐率就是1000次每秒操作。
    2. 最大化内存利用率：
        - 所有进程分配的虚拟内存的全部数量是受磁盘上交换空间的数量限制的。虚拟内存是一个有限的空间，必须 高效地使用。
    3. 关系：

- 最大化吞吐率与最大化利用率之间是互相牵制的。以堆利用率为代价，很容易编写出吞吐率最大化的分配器。分配器设计中一个有趣的挑战就是在两个目标之间找到一个适当的平衡。

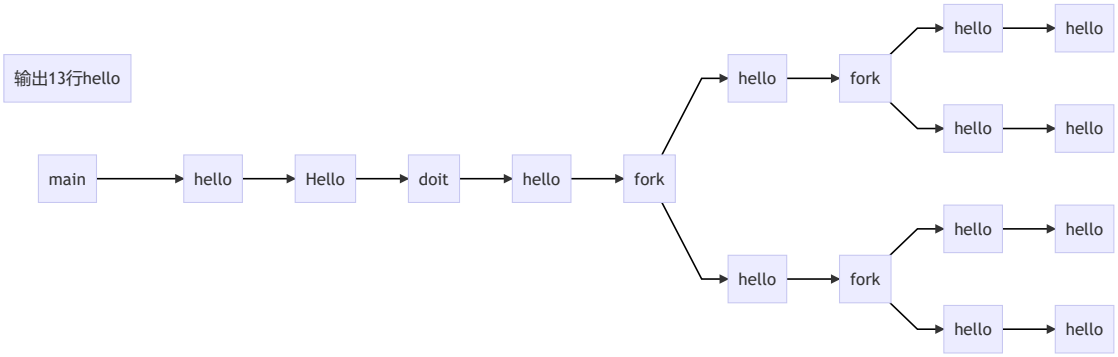
5. **典型的磁盘调度算法有哪些，请至少说出三种以上，并简单介绍其优缺点。**
    1. 最短寻道时间第一（Shortest Seek Time First)
        - 在最近的轨道上选择请求
        - 优缺点：
            - 饥饿
            - 陷入局部最优，达不到全局最优
    2. F-Scan (Elevator)
        - 由外到内，或内到外，一个方向扫完之后才处理新来的请求
        - 优缺点：
            - 避免饥饿
            - 效率不高
    3. C-Scan (Elevator)
        - 只是从外到内，再置位到最外道
        - 优缺点：
            - 所有磁盘更平
    4. 最短定位时间第一 (Shotest Positioning Time First)
        - 考虑到硬盘旋转的方向。寻道时间与旋转时间的问题
        - 优缺点：
            - 性能高
            - 实现较困难，操作系统通常不知道轨道边界在哪里或者磁头现在在哪里。

Process

1. 下列程程序会输出多少行 hello？请画图展示 fork 和输出的过程。

```
#include “csapp.h”
void doit() {
    printf("hello\n");
    fork();
    printf("hello\n");
    fork();
    printf("hello\n");
}

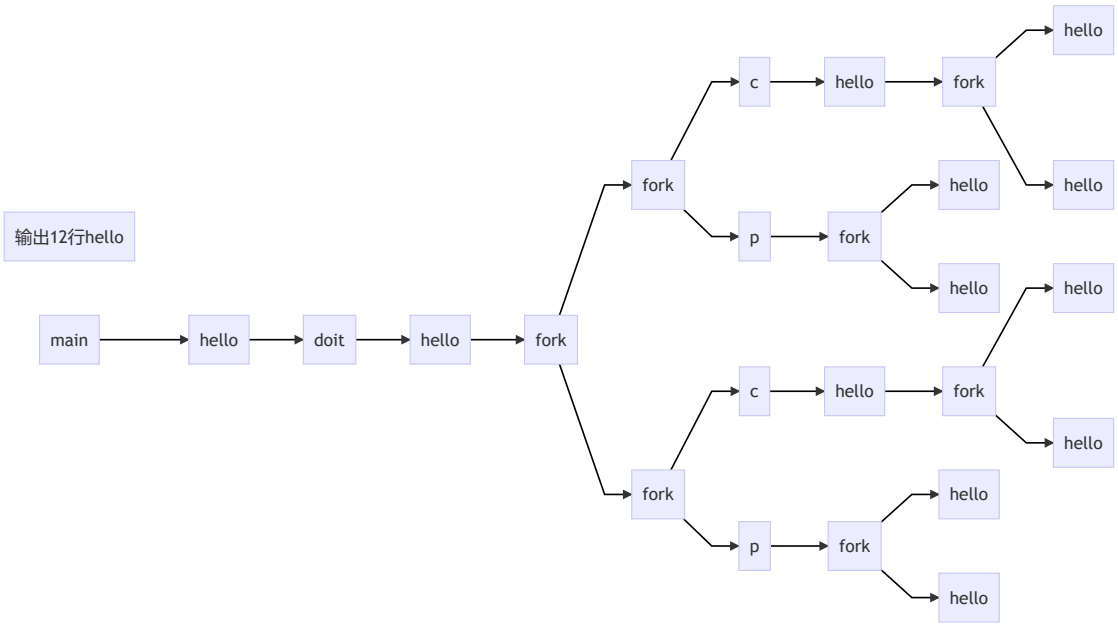
int main() {
    printf("hello\n");
    printf("hello\n");
    doit();
    printf("hello\n");
    exit(0);
}
```



2. 下列程程序会输出多少行 hello？请画图展示 fork 和输出的过程。

```
#include “csapp.h”
void doit() {
    printf("hello\n");
    fork();
    if(fork()==0)
    printf("hello\n");
    fork();
    printf("hello\n");
}

int main() {
    printf("hello\n");
    doit();
    exit(0);
}
```



Address Translation

1. 一台计算机中的内存访问需要先将虚拟地址（Virtual Address）转为物理地址（Physical Address），下面是该计算机系统进行地址转换时相关的一些参数：

- 主存的每个 byte 都有一个独立的地址；
- 主存每次访问的粒度是一个字节，该计算机的word定位为1 Byte（而不是4 Byte）；
- 系统采用单层页表；
- 虚拟地址由 25 bits 组成；
- PPN（Physical Page Number）是 9 bits；
- PTE（Page Table Entry）的总数量是 2^{14}.
- TLB 是 4 路组相连，一共能存储 16 个 entry。

1. 请填写下表：（1’* 4 = 4’）

VPO 中包含的 bit 数量	**11**
物理地址的 bit 数	20
页大小	2KB
TLBT（TLB Tag）的 bit 数	12

TLB 中的内容如下（数字为 16 进制）：

Set	Tag	PPN	Valid	Tag	PPN	Valid
0	0CD	09	1	AA1	00	1
	3E0	62	0	C4C	48	1
1	312	45	0	010	75	1
	987	3A	1	D39	3F	0
2	038	E3	0	0A7	13	1
	18B	52	1	49B	11	0
3	6C0	42	0	075	50	0
	013	39	1	0F2	0D	0

TLB: 4 sets, 16 entries, 4-way set associative

Page Table 中前 16 个 entry 的内容如下（数字为 16 进制）：

VPN	PPN	Valid	VPN	PPN	Valid
00	39	1	08	OD	0
01	52	1	09	45	0
02	E3	0	0A	13	1
03	00	1	0B	3A	1
04	11	0	0C	09	1
05	50	0	0D	42	0
06	62	0	0E	3F	0
07	75	1	0F	48	1

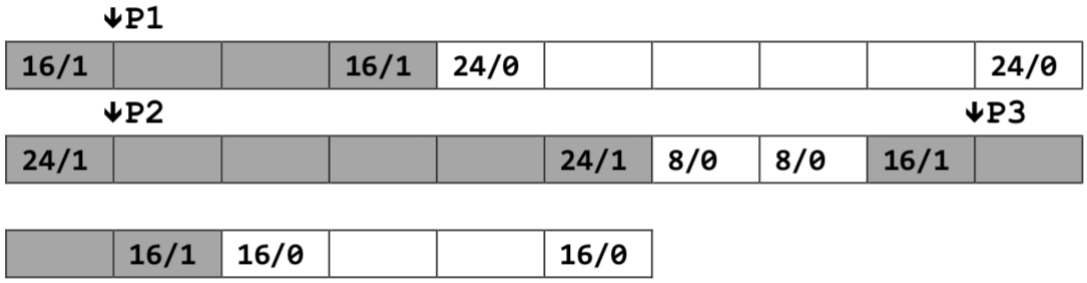
Page Table: Only the first 16 PTEs are shown

2. 请将以下虚拟地址转位物理地址，并将填写下表中各项（如果无法确定值，请填写--）。（1’* 14 = 14’）

Parameter	Value	Value
Virtual Address	0x014F2D3	0x07C01A5
VPN	0x029E	0x0F80
TLB Index	0x2	0x0
TLB Tag	0x0A7	0x3E0
TLB Hit? (Y/N)	Y	Y
Page Fault? (Y/N)	N	Y
PPN	0x13	-
Physical Address	0x09AD3	-

Memory Allocation（12 分）

下图表示计算机内的 heap 区，其中每个块表示 **1 个 word（4 个字节）**，灰色表示已经分配的块，白色表示空闲块，二者都有 Header 和 Footer，块中的 Header 和 Footer 已经在图中标记（各自占用 4 个字节）。分配器按照**双字对齐**。程序按照行 1 ~ 行 6 运行以下内存分配操作的请求序列（malloc()和 free()）。

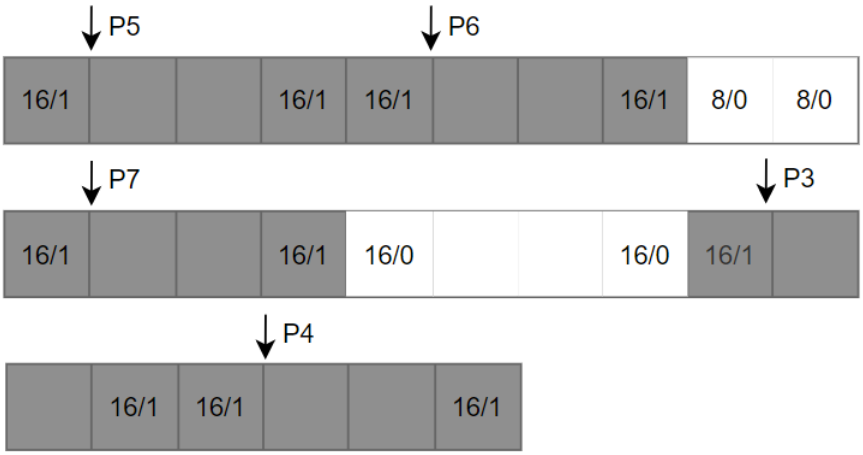



```
1. P4 = malloc(8);
2. free(P1);
3. P5 = malloc(6);
4. free(P2);
5. P6 = malloc(3);
6. P7 = malloc(2);
```

请回答以下问题。假设采用“**立即合并**”的策略，分配空间时，会按照用户要求分割 free block，剩下的空间空间还可能再次利用。（P1 和 P3 的用户负载均为 8 字节, P2 为 16 字节。）

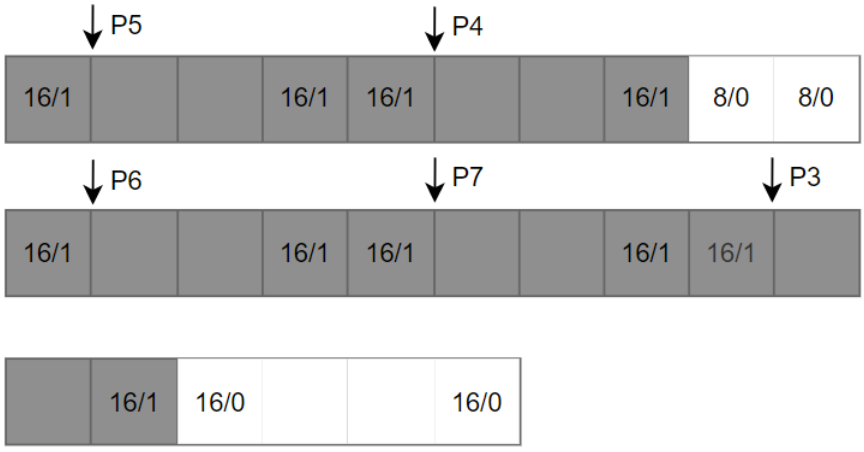
1. 假设采用 best-fit 策略来寻找空闲块。请画出执行完第 6 行代码后 heap 区内存的状态图，标记出来每个块的 header 和 footer 的值（3’）。请计算此时内部碎片的总大小是多少字节（3’）。

```
# payload + header + footer, double word alignment
1. P4: 8 + 4 + 4 = 16
2. free(P1)
3. P5: 6 + 4 + 4 = 14 => 16
4. free(P2)
5. P6: 3 + 4 + 4 = 11 => 16
6. P7: 2 + 4 + 4 = 10 => 16
```



内部碎片总大小为：8 + 16 = 24字节

2. 假设采用 first-fit 策略来寻找空闲块。请画出执行完第 6 行代码后 heap 区内存的状态图，表基础来每个块的 header 和 footer 的值（3’）。请计算此时内部碎片的总大小是多少字节（3’）。



内部碎片总大小为：8 + 16 = 24 字节

I/O 调度（12分）

计算机每个扇区为 512B。以下 I/O 请求几乎同时、但按照顺序到达 I/O 调度层，其中的数字为扇区编号。当前磁头正处于 100 号扇区处，并向扇区编号较大的方向移动。

I/O 请求序列为{120，90，45，170，50，75，125，30，20}

假设计算机只有一块磁盘，没有进行 RAID 配置，所使用的磁盘调度策略分别为 First Come First Serve (FCFS)、Shortest Seek Time First (SSTF)、F-SCAN(磁头任意移动方向都可以服务 I/O 请求)、C-SCAN 调度策略(只有磁头从小编号扇区向大编号扇区移动的过程中可以服务 I/O 请求)。

请分别给出四种策略的实际I/O请求服务序列，以及完成这些I/O请求的总寻道时间（假设每移动一个扇区，寻道时间加 1）。对于 SSTF，如果遇到寻道时间相同的请求，尽可能不改变磁头移动方向。

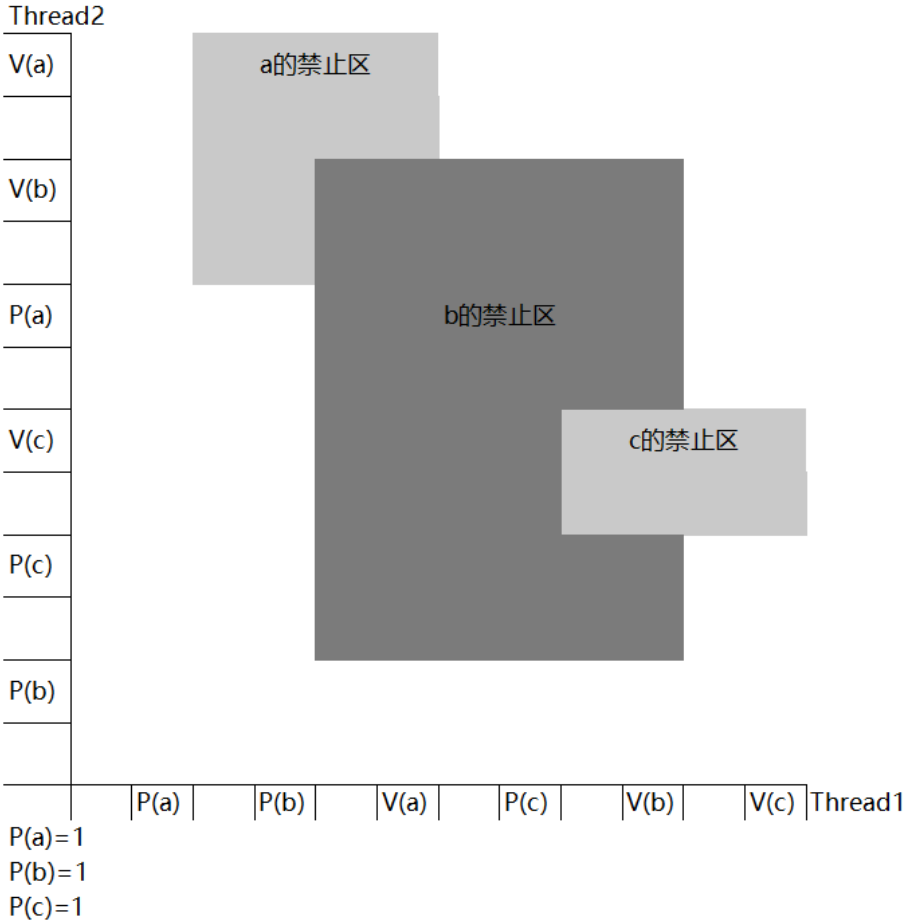
- FCFS
  - ```
    100 - 120 - 90 - 45 - 170 - 50 - 75 - 125 - 30 - 20
    20 + 30 + 45 + 125 + 120 + 25 + 50 + 95 + 10 = 520
    ```
    ```

- SSTF
  - ```
    100 - 90 - 75 - 50 - 45 - 30 - 20 - 120 - 125 - 170
    10 + 15 + 25 + 5 + 15 + 10 + 100 + 5 + 45 = 230
    ```
- F-SCAN
  - ```
    # 并向扇区编号较大的方向移动（由外向内）
    100 - 120 - 125 - 170 - 90 - 75 - 50 - 45 - 30 - 20
    20 + 5 + 45 + 80 + 15 + 25 + 5 + 15 + 10 = 220
    ```
- C-SCAN
  - ```
    # 只有磁头从小编号扇区向大编号扇区移动的过程中可以服务 I/O 请求
    100 - 120 - 125 - 170 - 20 - 30 - 45 - 50 - 75 - 90
    20 + 5 + 45 + 150 + 20 + 15 + 5 + 25 + 15 = 300
    ```

Deadlock（10’）

如下表所示，有两个线程 Thread 1 和 Thread 2 同时工作，a, b, c 为信号量，初始值均为 1。请问 Thread 1 和 Thread 2 是否会发生死锁？请画出进程图，并作出解释。

Thread	**Thread 1**	**Thread 2**
step1	P(a)	P(b)
step2	P(b)	P(c)
step3	V(a)	V(c)
step4	P(c)	P(a)
step5	V(b)	V(b)
step6	V(c)	V(a)
    ```



会发生死锁
因为a的禁止区与b的禁止区的重叠区域引起了一组死锁区域的状态，所以会发生死锁

线程安全（12’）

请判断下面每个函数的类型是线程安全函数、线程不安全函数，还是是可重入函数。

```
int t;
void f(int *x, int *y) {
    t = *x + *y;
}

void g(int*x, int *y) {
    static int m = *x;
    *y = *x;
    *x = m;
}

void h(int *x, int *y) {
    P(&mutex);
    t = *x;
    *x = *y;
    *y = t;
    V(&mutex);
}

void k(int *x, int *y) {
```

```
    P(&mutex);
    t = *x;
    V(&mutex);
    *x = *y;
    P(&mutex);
    *y = t;
    V(&mutex);
}

void m(int *x, int *y) {
    int t = *x;
    *x = *y;
    *y = t;
}

void n(int *x, int *y) {
    *x = *x ^ *y;
    *y = *x ^ *y;
    *x = *x ^ *y;
}
```

```
f: 线程不安全函数
g: 线程不安全函数
h: 线程安全函数
k: 线程安全函数
m: 线程不安全函数
n: 可重入函数
```

File System（20分）

假设文件系统采用类似 Very Simple File System 的方式进行数据分布，包括 superblock, inode bitmap, data bitmap, inodes （每个 inode 包含 12 个 direct pointers，2 个 indirect point，1 个 double indirect pointer，所有指针均为 4B）和 data block，每个文件或目录每次至少分配一个 block。Superblock, inode bitmap 和 data bitmap 各占一个 block，每个 block 的大小为4KB。

假设以下两问中的操作是独立进行的，初试状态缓存中没有任何数据，访问一次即缓存；假设对文件的访问或修改需要在该文件、该文件所在文件夹记录访问/修改时间（再上层目录不受影响），所有修改会立刻落盘（对一个数据块的多次更新算消耗一次 I/O）。请计算下列操作需要进行多少次 I/O（包括读、写）？建议写出具体过程，每个部分的 I/O 次数，以便出错时能够得一部分分数。

1. 文件系统中已有/home/ics/test 文件，大小为 2MB，读出该文件的全部内容。（10’）

```
1. read S
2. read / inode
3. read / data block
4. read /home inode
5. read /home data block
6. read /home/ics inode
7. read /home/ics data block
8. read /home/ics/test inode
9. read /home/ics/test data block 2*1024KB/4KB=512, 512 < 1024, need 1 indirect
pointer
10. write /home/ics/test inode, update access time
Total 512 + 1 + 1 + 8 = 522 I/O
```

1. 文件系统中已有/home/ics/test 文件，大小为 10MB，读出该文件的全部内容。（9’）(A卷)

```
1. read S
2. read / inode
3. read / data block
4. read /home inode
5. read /home data block
6. read /home/ics inode
7. read /home/ics data block
8. read /home/ics/test inode
9. read /home/ics/test data block 10*1024KB/4KB=2560, 2560 > 2*1024, need 2
indirect pointer and 1 double indirect pointer
10. write /home/ics/test inode, update access time
Total 2560 + 2 + 1 + 1 + 8 = 2572 I/O
```

2. 文件系统中已有/home/ics/test 文件，大小为 2MB，在文件尾部追加写入 30MB 数据。（10’）

```
1. read S
2. read / inode
3. read / data block
4. read /home inode
5. read /home data block
6. read /home/ics inode
7. read /home/ics data block
8. read /home/ics/test inode
9. read /home/ics/test data bitmap to find free blocks
10. 30*1024KB/4KB=7680, 7680 < 2*1024, need 2 indirect pointer and 1 indirect
pointer
11. write /home/ics/test inode, update access time
Total 7680 + 2 + 1 + 1 + 8 = 7692 I/O
```